

Doubly Linked List Data Structure

 <https://github.com/learn-co-curriculum/python-p3-dsa-doubly-linked-list>  <https://github.com/learn-co-curriculum/python-p3-dsa-doubly-linked-list/issues/new>

Learning Goals

- Identify the use cases for a **doubly linked list**.
- Demonstrate common methods for a doubly linked list.
- Differentiate between a doubly linked list and a singly linked list.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Stack** is a linear data structure that follows the principle of Last In First Out (LIFO).
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

We learned in the last lesson about singly linked lists, their use cases, and the general concept behind linked lists. In this lesson, we're going to dive into what a doubly linked list is, and what the difference is between singly and doubly linked lists.

Defining a Doubly Linked List

In the previous lessons we learned how unlike lists, linked lists are not indexed, and in order to search from node to node, we need use pointers to go from one node onto the next node. But what if we wanted to go back a node? In a singly linked list, a node doesn't know which node came before it, because it doesn't have a pointer pointing to the previous node. Doubly linked lists have pointers to the next node *as well* as the previous node. Let's take a look at how we would build this out in our `Node` class:

```
class Node:
    def __init__(self, data, next_node = None, prev_node = None):
        self.data = data
        self.next_node = next_node
        self.prev_node = prev_node
```

All we really had to do was add a `prev_node` attribute, and now we have two pointers on our `Node` class, so that each node points in two directions: to the next node in the list, and to the previous node. While this is a really small and easy change to make to the structure of our node, by doing this change we are able to make our linked list much more useful, efficient, and dynamic!

Singly vs Doubly Linked Lists

One way we can improve the time complexity of our singly linked list implementation is by adding additional references to nodes in the list. For example, consider the following `SinglyLinkedList` class:

```
class SinglyLinkedList:

    def __init__(self, head = None):
        self.head = head
```

```
def append(self, node):
    # Add element to the beginning of the list if the list is empty
    if self.head == None:
        self.head = node
        return
    # Otherwise, traverse the list to find the last node
    last_node = self.head
    while last_node.next_node:
        last_node = last_node.next_node
    # and add the node to the end
    # 1 -> 2 -> 3
    last_node.next_node = node
    # 1 -> 2 -> 3 -> 4
```

The time complexity of its `append` method is $O(n)$, since we need to traverse each element of the linked list in order to find the last node and remove it. We can make the `append` method efficient by keeping track of the list's **tail** node in addition to the head:

```
class SinglyLinkedList:

    def __init__(self, head = None, tail = None):
        self.head = head
        self.tail = tail

    def append(self, node):
        # Add element to the beginning of the list if the list is empty
        if self.head == None:
            self.head = node
            self.tail = node
            return
        # no need to traverse! we can access the last node directly (self.tail)
        # 1 -> 2 -> 3
```

```

self.tail.next_node = node
# 1 -> 2 -> 3 -> 4
# we also need to make sure to keep track of the new tail

self.tail = node

```

After this refactor, the time complexity of our **append** method is $O(1)$, since we no longer need to traverse the list in order to find the tail before appending a new node. The tradeoff to keeping references to additional nodes in our list, like the tail, is it takes more space to keep track of these additional references. Adding additional reference can also increase the written complexity of our code for certain methods, since we need to maintain those references correctly.

A **doubly linked list** makes insertion and removal more efficient in certain cases by keeping references to the **previous** node in addition to the next node.

Let's say we have a singly linked list, and we wanted to remove the last item. We would have to traverse the *entire* list in order to find the *second to last node* in the list and assign it as the new tail, since we can't go directly to the tail and work backwards:

```

class SinglyLinkedList:

    def __init__(self, head = None, tail = None):
        self.head = head

    def delete_tail(self):
        if self.head == None:
            return
        # traverse the entire list to find the second-to-last node (prev)
        curr = self.head
        prev = None
        while curr.next_node:
            prev = curr
            curr = curr.next_node
        # remove the last node by removing the link between the second-to-last node and the tail
        # 1 -> 2 -> 3

```

```
prev.next_node = None
# 1 -> 2
```

In this case, the time complexity for removing a node from the end of the list is $O(n)$ since we need to traverse the list to find the second-to-last node.

With a **doubly linked list**, we already have a pointer on each node pointing to the previous node, so we can just take one step backwards from the **tail** by using **.prev_node** without needing to iterate:

```
class DoublyLinkedList:

    def __init__(self, head = None, tail = None):
        self.head = head

    def delete_tail(self):
        if self.head == self.tail:
            self.head = None
            self.tail = None
            # traverse the entire list to find the second-to-last node (prev)
        else:
            # access the second-to-last node (self.tail.prev_node)
            prev = self.tail.prev_node
            # update the tail and next_node pointers
            # 1 <-> 2 <-> 3

            prev.next_node = None
            self.tail = prev
            # 1 <-> 2
```

After this refactor, our time complexity for removing a node from the end of the list is $O(1)$, since we don't need to traverse the entire list to find the new tail.

The tradeoff is that we now need to maintain these references between nodes correctly for *all* our linked list methods. For example, the `append` method for a doubly linked list is more complicated than for a singly linked list, since we have to keep track of the `next_node` and `prev_node` correctly any time a node is added:

```
class DoublyLinkedList:

    def __init__(self, head = None, tail = None):
        self.head = head
        self.tail = tail

    def append(self, node):
        if self.head == None:
            self.head = node
            self.tail = node
            return
        node.prev_node = self.tail
        self.tail.next_node = node
        self.tail = node
```

Conclusion

Singly linked lists and doubly linked lists are very similar. Unlike lists, they are not indexed, and they use pointers to reference their nodes. Singly linked lists are one-directional, while doubly linked lists go both ways. Having the extra `prev_node` pointer in a doubly linked list can be useful in the following scenarios:

- Removing an item from the end of a list.
- Reversing the list (traversal from tail to head).
- Implementing "previous/next" operations or "undo/redo" operations (like building a playlist or a text editor).

The trade-offs are that a doubly linked list takes up more memory than a singly linked list, since we have to keep track of multiple pointers on each node; and the code for a doubly linked list is also more complicated to write and maintain because of the added complexity of keeping both

the `next_node` and `prev_node` references.

Resources

- [Linked List](https://en.wikipedia.org/wiki/Linked_list)  (https://en.wikipedia.org/wiki/Linked_list)